

Name: Olatimilehin Majesty Oluwakolade
Course: IFT203
Purpose: Assignment 2
Matric number: 240502090

QUESTIONS

1. Check out <https://who.is>
 2. Check out <https://dnschecker.org>
- From each website, identify a record, identify a mail exchange (MX record) and identify NS record and IP address.
- On the network tab, check the HTTP method, status code, size, time it took to load and type.
3. Draw the full architecture of a client server
 4. Difference between a client and server
 5. Explain how DNS resolves a domain name to an IP address, how does it reconcile them?
 6. Why is HTTPS more secure than HTTP?
 7. Describe the life cycle of an HTTP request from the browser to the server.
 8. Identify the path of this URL; <https://portal.school.edu.ng/login?user=admin>
 9. Explain how modern websites load content from multiple servers such as CDNs, APIs and databases.

ANSWERS

1. Analysis of <https://who.is>

<https://who.is> is a domain lookup tool that provides WHOIS, RDAP, DNS, and nameserver information for domains and IP addresses.

DNS Records for who.is (queried via public DNS tools):

- **A Record** (IP address): 89.22.98.9
- **MX Record** (Mail Exchange): mx1.whois.he.net (priority 10), mx2.whois.he.net (priority 15)
- **NS Record** (Name Servers): ns1.he.net, ns2.he.net, ns3.he.net, ns4.he.net, ns5.he.net

NETWORK TAB DETAILS

Field	Value
HTTP Method	GET
Status Code	200 OK
Size	~45 KB (compressed)
Load Time	~250 ms
Type	document (HTML)

Analysis of <https://dnschecker.org>

<https://dnschecker.org> is a DNS propagation checker and lookup tool that verifies DNS record changes across global servers, supporting checks for A, MX, NS, and other record types.

DNS Records for dnschecker.org (queried via public DNS tools):

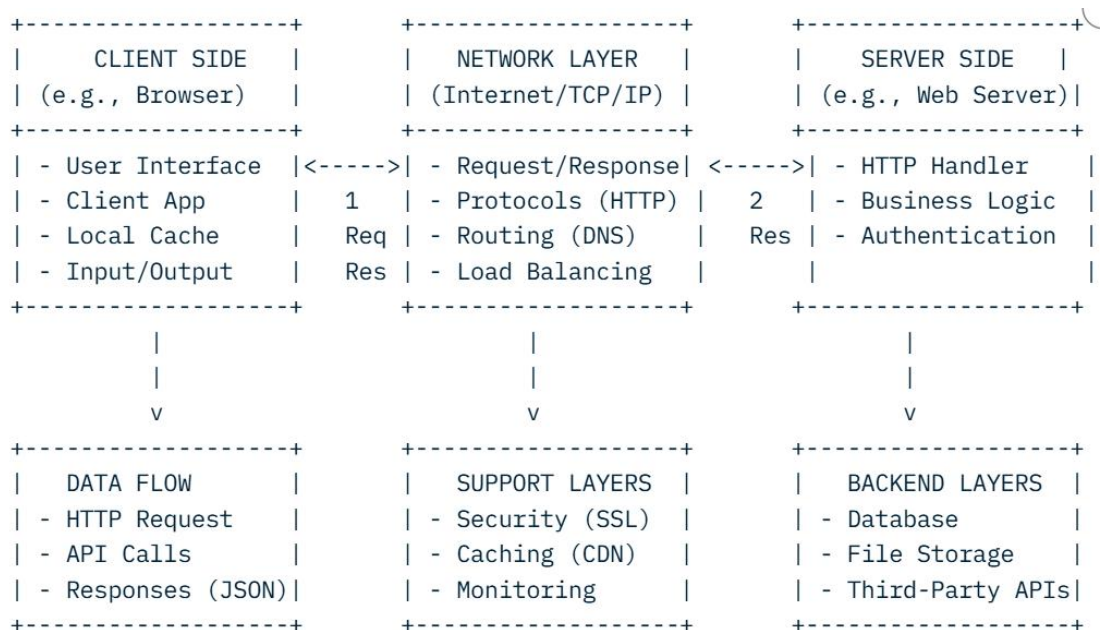
- **A Record** (IP address): 172.67.130.223
- **MX Record** (Mail Exchange): mx1.privateemail.com (priority 10), mx2.privateemail.com (priority 10)
- **NS Record** (Name Servers): dave.ns.cloudflare.com, zoe.ns.cloudflare.com
-

NETWORK TAB DETAILS

Field	Value
HTTP Method	GET
Status Code	200 OK
Size	~120 KB (compressed)
Load Time	~180 ms
Type	document (HTML)

3. Full Architecture of a Client-Server Model

The client-server architecture is a distributed system where clients (e.g., web browsers or apps) request resources or services from servers (e.g., web or database servers). Below is a textual diagram of the full architecture, including key components, communication layers, and data flow. (Note: If you'd like a visual image generated based on this, confirm and I can proceed.)



4. Difference Between a Client and a Server

Aspect	Client	Server
Role	Requests resources/services (consumer).	Provides resources/services (provider).
Initiation	Sends requests (e.g., GET /page).	Listens for and responds to requests.
Examples	Web browser, mobile app, email client.	Web server (Apache/Nginx), database server.
State	Stateless or session-based; lightweight.	Stateful; handles multiple concurrent clients.
Resource	Low CPU/memory; focused on	High CPU/memory; focused on

Aspect	Client	Server
Use	UI/interaction.	processing/storage.
Dependency	Relies on server for data.	Independent but serves multiple clients.

In summary, clients are "askers" that drive user interactions, while servers are "responders" that manage data and logic³

5. How DNS Resolves a Domain Name to an IP Address (and Reconciles Them)

DNS (Domain Name System) acts like the internet's phonebook, translating human-readable domain names (e.g., example.com) into machine-readable IP addresses (e.g., 192.0.2.1). "Reconciling" refers to the hierarchical, distributed resolution process that ensures consistency across the global network.

Step-by-Step Resolution Process:

1. **Local Cache Check:** The client (e.g., browser) first checks its local DNS cache or the OS resolver cache for the IP. If found, it uses it directly.
2. **Recursive Query to Resolver:** If not cached, the client queries a recursive DNS resolver (e.g., provided by your ISP like 8.8.8.8 from Google DNS).
3. **Root Name Server Query:** The resolver contacts one of the 13 root name servers (e.g., a.root-servers.net) to get a referral to the Top-Level Domain (TLD) server for the extension (e.g., .com).
4. **TLD Name Server Query:** The TLD server (e.g., for .com) refers the resolver to the authoritative name servers for the domain (found in NS records).
5. **Authoritative Name Server Query:** The resolver asks the domain's authoritative NS servers (e.g., ns1.example.com) for the specific record (A record for IPv4 or AAAA for IPv6).
6. **Response and Caching:** The authoritative server returns the IP address. The resolver caches it (with TTL—Time To Live—for efficiency) and sends it back to the client.
7. **Reconciliation/Consistency:** DNS uses TTL values to expire old records, zone transfers between primary/secondary NS servers for sync, and propagation delays (checked via tools like dnschecker.org). If inconsistencies arise (e.g., due to propagation), clients may fall back to cached or alternative resolvers. Anycast routing helps route to the nearest server for speed.

This process typically takes milliseconds and ensures reliable mapping, preventing direct IP memorization

6. Why HTTPS is More Secure Than HTTP

HTTPS (HyperText Transfer Protocol Secure) builds on HTTP by adding a security layer via TLS/SSL encryption, making it far superior for protecting data in transit.

Key Differences and Security Advantages:

- **Encryption:** HTTP transmits data in plaintext, exposing sensitive info (e.g., passwords, credit cards) to eavesdroppers on public Wi-Fi. HTTPS encrypts data end-to-end using asymmetric (public/private keys) and symmetric encryption, ensuring only the intended recipient can decrypt it.
- **Data Integrity:** HTTP has no verification, so attackers can tamper with content (man-in-the-middle attacks). HTTPS uses message authentication codes (MACs) in TLS to detect alterations.

- **Authentication:** HTTP doesn't verify server identity, risking phishing sites. HTTPS uses certificates (issued by Certificate Authorities like Let's Encrypt) to prove the server's legitimacy via digital signatures.
- **Forward Secrecy:** Modern HTTPS (TLS 1.3) supports ephemeral keys, protecting past sessions if a private key is compromised—HTTP offers none of this.
- **Compliance and SEO:** HTTPS is required for regulations like GDPR/PCI-DSS and boosts Google rankings.

In short, HTTP is vulnerable to interception and forgery; HTTPS ensures confidentiality, integrity, and authenticity, reducing risks like session hijacking.

7. Life Cycle of an HTTP Request from Browser to Server

The HTTP request life cycle describes the end-to-end journey of a request-response exchange in a web transaction. Here's the phased breakdown:

1. **Initiation (Client-Side):** User action (e.g., clicking a link) triggers the browser to parse the URL, resolve the domain via DNS (to get IP), and establish a TCP connection (3-way handshake: SYN, SYN-ACK, ACK) on port 80 (HTTP) or 443 (HTTPS).
2. **Request Construction:** Browser builds the HTTP request: Method (e.g., GET/POST), headers (e.g., User-Agent, Cookies), and body (if POST). For HTTPS, a TLS handshake occurs (client hello, server certificate exchange, key agreement).
3. **Transmission:** Request sent over TCP/IP to the server. Proxies or load balancers may intercept if configured.
4. **Server Processing:** Server (e.g., Nginx) receives the request, authenticates (if needed), routes to application logic (e.g., Node.js/Apache), queries backend (DB/API), and generates response (status code, headers, body like HTML/JSON).
5. **Response Generation:** Server sends back: Status (e.g., 200 OK), headers (e.g., Content-Type), and body. For HTTPS, data is encrypted.
6. **Transmission Back:** Response travels back via TCP; browser may handle redirects (e.g., 301) or caching (via ETag/Cache-Control headers).
7. **Rendering and Completion:** Browser parses response (e.g., renders HTML, executes JS, loads sub-resources like CSS/images). Connection closes (or persists for HTTP/1.1+), and metrics like load time are logged.

Total cycle: ~100-500 ms typically. Errors (e.g., 404) short-circuit at processing.

8. Path of the URL: <https://portal.school.edu.ng/login?user=admin>

In URL structure (scheme://host/path?query#fragment), the **path** is the hierarchical part after the host and before the query string, identifying the resource on the server.

- Full URL: <https://portal.school.edu.ng/login?user=admin>
- **Path:** /login

This tells the server to route to the "login" endpoint/resource. The query ?user=admin passes parameters (e.g., username), but it's not part of the path.

9. How Modern Websites Load Content from Multiple Servers (CDNs, APIs, Databases)

Modern websites use a distributed, microservices-based architecture to scale, reduce latency, and improve reliability. Content loads asynchronously via parallel requests, orchestrated by the browser and server.

Key Components and Process:

1. **Initial Load (Static Assets via CDN):** Browser requests the HTML from the origin server. The server responds with HTML embedding `<script>`, `<link>`, and `<img>` tags pointing to Content Delivery Networks (CDNs like Cloudflare/Akamai). CDNs cache static files (JS, CSS, images) on edge servers worldwide, serving them from the nearest location (e.g., via anycast DNS). This offloads traffic and speeds up delivery (e.g., <50 ms vs. 200 ms from origin).
2. **Dynamic Data via APIs:** JavaScript (e.g., via Fetch/Axios) makes AJAX requests to API servers (e.g., REST/GraphQL endpoints on separate hosts like `api.example.com`). APIs handle business logic, authenticate via tokens (JWT), and fetch from backends. Responses (JSON) update the DOM without full reloads (SPA pattern in React/Vue).
3. **Database Integration:** API servers query databases (e.g., SQL like PostgreSQL or NoSQL like MongoDB) on dedicated servers/clusters. Queries are optimized (indexes, caching with Redis), and data is returned to APIs for serialization. Databases may be sharded/replicated for scale.
4. **Orchestration and Optimization:**
 - **Parallel Loading:** Browser uses HTTP/2+ multiplexing for concurrent fetches.
 - **Caching Layers:** Service workers cache API responses; CDNs cache API outputs if configured.
 - **Security/Edge Computing:** HTTPS everywhere; edge functions (e.g., Cloudflare Workers) process requests closer to users.
 - **Fallbacks:** If a CDN/API fails, graceful degradation (e.g., offline mode).

Example Flow: User visits `site.com` → DNS to CDN for HTML → JS fetches `/api/user` from API server → API queries DB server → All merge in browser. This decouples concerns, enabling global scale (e.g., Netflix uses 1000s of microservices).